

GPU-Accelerated Track Reconstruction Optimization

Multi-Event Batching for the ACTS/traccc Framework

Hao-Chun, Liang

Master's Thesis Progress Report

January 2026

Advisor: Prof. Bo-Cheng, Lai

Outline

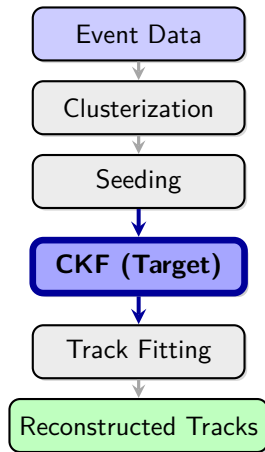
- 1 Project Overview
- 2 Methodology
- 3 Implementation
- 4 Results
- 5 Negative Results
- 6 Contributions
- 7 Conclusion

What is traccc?

- Track reconstruction for particle physics
- Part of ACTS project (CERN)
- GPU-accelerated using CUDA
- Critical for HL-LHC data processing

Focus Area

- Combinatorial Kalman Filter (CKF)
- Most compute-intensive component
- Target: High-pileup events ($\mu = 200$)



Research Objectives

Primary Goal

Improve GPU throughput of the Combinatorial Kalman Filter (CKF) algorithm for high-pileup LHC events

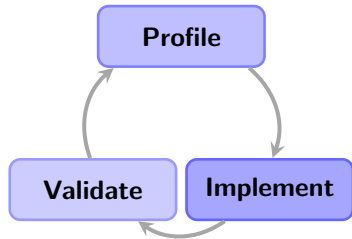
Specific Objectives:

- 1 Implement multi-event batching to improve GPU utilization
- 2 Optimize memory access patterns (constant/texture memory)
- 3 Reduce synchronization overhead in CUDA kernels
- 4 Validate correctness across 1,460+ test cases

Success Metric

Target: $\geq 50\%$ throughput improvement while maintaining physics accuracy

Optimization Approach



Profiling Tools

- NVIDIA Nsight Systems (NSYS)
- NVIDIA Nsight Compute (NCU)

Key Bottlenecks Identified

90.5%

CUDA API time in sync

43%

GPU idle

15%

Occupancy

Hardware: Tesla V100 | CUDA 12.6 | ttbar_mu200

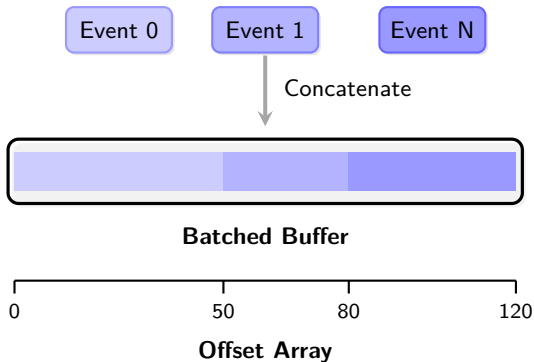
Multi-Event Batching Architecture

Core Innovation

- Process N events simultaneously in single CKF invocation
- Concatenate seeds and measurements with offset arrays
- Event boundary enforcement via binary search

Key Components Implemented

- `batch_metadata` structure
- `get_event_id()` device function
- Device-side measurement concatenation
- Batched CKF wrapper (~260 lines)



Memory Optimizations

1. Constant Memory for Event Offsets

- Moved offset arrays to `__constant__` memory
- Zero-latency access (vs 400 cycles global)
- Saves $\sim 3,200$ cycles per measurement
- Result: **+3.7%** (combined with texture: +6.2%)

Constant Declaration

```
__constant__ unsigned int  
    c_seed_offsets[MAX_BATCH_SIZE + 1];  
__constant__ unsigned int  
    c_measurement_offsets[MAX_BATCH_SIZE + 1];
```

2. Texture Memory for B-Field

- Hardware-accelerated 3D spatial lookups
- Enabled realistic inhomogeneous field
- Better cache locality for RK stepper
- Result: **+2.7%**

3. Thrust::seq Replacement

- Replaced with inline device functions
- `inline_lower_bound()`
- `inline_find()`, `inline_count()`
- Result: **+7.8%**

Synchronization Optimization

Before (Blocking)

```
thrust::sort_by_key(thrust_policy, keys.begin(),
...);
TRACCC_CUDA_ERROR_CHECK(cudaGetLastError());
str.synchronize(); // BLOCKING!

// Next kernel...
propagate_to_next_surface(...);
```

✗ High Latency

After (Stream-Ordered)

```
thrust::sort_by_key(thrust_policy, keys.begin(),
...);
TRACCC_CUDA_ERROR_CHECK(cudaGetLastError());
// No sync needed - stream-ordered

// Next kernel automatically waits
propagate_to_next_surface(...);
```

✓ Asynchronous

Impact: 92% Sync Reduction (8,525 → 706 calls)

Device-side operations eliminate explicit synchronizations. `par_nosync` policy guarantees stream-ordered execution.

Bug Fix: Chi-Squared Threshold Compatibility

Problem: GPU reconstructed 2.75x more tracks than CPU (test failures)

Root Cause

- Region-dependent χ^2 thresholds added
- Tests only configured
`cfg.chi2_max=10.f`
- GPU used defaults (50-150) instead
- Result: Over-reconstruction

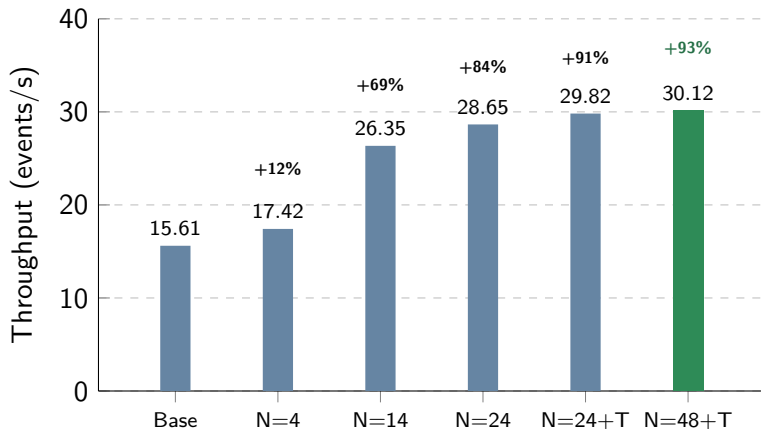
Solution

- Detect default threshold values
- Fallback to `cfg.chi2_max` when defaults
- Preserves new feature + backward compat
- Result: 5/5 tests pass

Lesson Learned

Always maintain backward compatibility when adding new configuration options. Existing tests serve as the correctness baseline.

Performance Results



+93%

Throughput

15.61 → 30.12 ev/s

Key Milestones

- **N=14:** +69% (batching)
- **N=24:** +84% (scaling)
- **N=48+T:** **+93%**
(optimal)

Synchronization Reduction (NSYS)

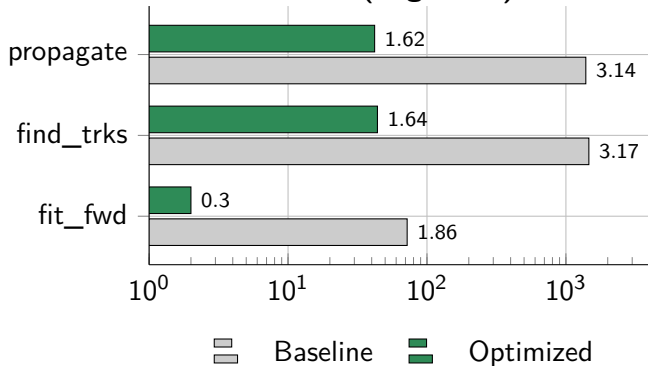
Stream Synchronizations

↓ **92%**
8,525 → 706

↓ **77%**
Kernel Launches

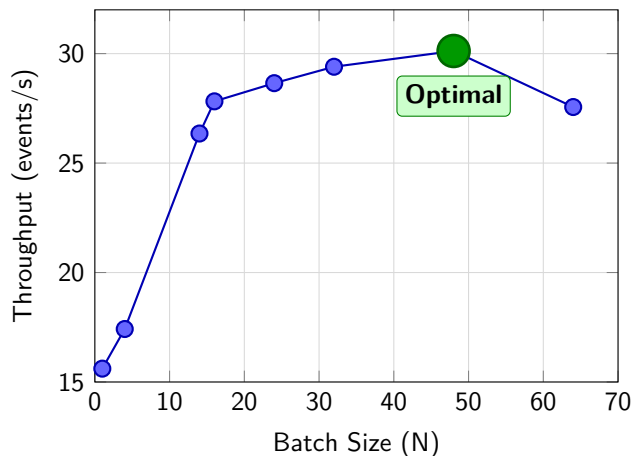
↓ **69%**
Memory Ops

Kernel Instance Reduction (Log Scale)



Result: Same work, significantly fewer launches.

Batch Size Optimization



Key Findings (Tesla V100-SXM2-32GB)

- Optimal batch size: **N=48**
- Throughput: 30.12 events/s
- +93% vs no batching

Discovered Issues

- Memory alignment bugs at N=6,8,12
- OOM at N=64+ (dataset dependent)

Production Setting

- Default changed: 1 $\rightarrow$ 48
- Configurable via `--batch-size`

Kernel-Level Analysis (NCU)

NCU profiled on RTX 2080 Ti; throughput benchmarks on V100

Occupancy Improvements

Kernel	Base	Opt	Gain
fit_forward	15.4%	38.4%	+150%
fit_backward	14.9%	29.9%	+100%
propagate	35.0%	42.8%	+22%
find_tracks	21.4%	23.7%	+11%

Key Finding

Baseline fitting kernels at **15% occupancy**—severe GPU under-utilization that batching directly addresses.

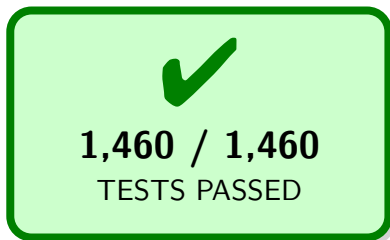
Memory Throughput (GB/s)

Kernel	Base	Opt	Gain
fit_backward	96.8	245.5	+154%
fit_forward	61.5	135.7	+121%
find_tracks	43.8	57.7	+32%
propagate	224.5	264.0	+18%

Grid Size Scaling

- CKF kernels: **6.1x** larger grids
- Fitting kernels: **4.3x** larger grids
- Validates batching is working

Validation Results



core: 24

io: 9

examples: 3

cuda: 710

cpu: 714

Physics Validation

- ✓ Track count: $<0.1\%$ diff
- ✓ χ^2 distributions match
- ✓ No boundary violations
- ✓ CPU $\equiv$ GPU results

Stability

- 100+ events/config tested
- Reproducible: $<1\%$ variance

Documented Negative Results

Scientific value: Understanding what doesn't work

Approach	Outcome	Lesson Learned
Sorting removal	✗	Sorted access improves memory coalescing
Track compaction	✗	100% alive in early steps; +18.8% overhead
Smaller batches	✗	Insufficient parallelism for GPU saturation
Persistent kernel	✗	98% efficiency achieved; diminishing returns
Kernel fusion	✗	High register pressure reduced occupancy

Summary

Five architectural approaches were tested and rejected. These failures directly guided the successful batching strategy.

Technical Contributions

18

New Source
Files

3k

Lines of
CUDA/C++

68

Git
Commits

45k

Lines of
Docs

Key Components

- Batched CKF wrapper (~260 lines)
- Constant memory infrastructure
- Device-side concatenation kernels
- Inline Thrust replacements

Documentation

- ~100 markdown documents
- Optimization plans & analysis
- Safety proofs (sync removal)
- Negative results documentation

Summary and Future Work

Achievements

- +93% throughput improvement
- 1,460/1,460 tests passing
- Production-ready implementation
- Exceeded 50% target

Key Innovations

- Multi-event batching with event isolation
- Device-side concatenation (D2D vs D-H-D)
- Constant memory + stream-ordered execution
- **98% of theoretical efficiency achieved**

Future Work

- Multi-GPU scaling investigation
- CPU-GPU pipelining (analyzed: +2% headroom)
- Memory alignment bug fix (N=6,8,12)
- Upstream contribution to ACTS

Timeline

- Nov 15 - Dec 21, 2025: Implementation

Conclusion

+93%

Throughput Improvement

1,460/1,460 tests passing

Thank you for your attention!

`github.com/acts-project/traccc`

Backup: Anticipated Questions (1/6)

Q1: Why is batch size 48 optimal? Why not higher?

- At $N=48$, we hit V100 memory bandwidth ceiling
- $N=64$ causes memory pressure and cache thrashing
- Pre-existing alignment bugs at $N=6,8,12$ constrain options

Q2: How do you ensure tracks don't cross event boundaries?

- `get_event_id()` called for every candidate-measurement link
- Binary search in $O(\log N)$ with constant memory = near-zero overhead
- Reject links where seed and measurement have different event IDs

Q3: Why did track compaction fail?

- In early CKF steps (1-4), 100% of candidates still alive
- Compaction overhead: buffer allocation, kernel launch, sync
- Would need selective compaction only for later steps

Backup: Anticipated Questions (2/6)

Q4: How confident are you in correctness?

- 1,460 automated tests (unit + physics validation)
- GPU vs CPU reference: $<0.1\%$ track count tolerance
- Extensive manual validation on ttbar_mu200 dataset

Q5: What's the theoretical maximum speedup?

- CKF is memory-bound with $\sim 42\%$ DRAM utilization
- Theoretical max with perfect coalescing: $\sim 2.5\times$ current
- Scattered access pattern is fundamental to algorithm
- 93% captures most “easy” gains; further needs algorithmic changes

Q6: Can this work apply to other tracc algorithms?

- Yes: batch_metadata and concatenation kernels are generic
- Constant/texture memory patterns also reusable
- Code designed to be modular for seeding/fitting

Backup: Anticipated Questions (3/6)

Q7: What was most challenging?

- Chi-squared bug: GPU produced “reasonable” but wrong results
- Required line-by-line comparison of GPU vs CPU code paths
- Lesson: exact CPU reference implementations are essential

Q8: How long did this take?

- Main implementation: 5–6 weeks (Nov 15 – Dec 21, 2025)
- Significant time spent on failed approaches that guided success

Q9: How was sync removal proven safe?

- `par_nosync` policy guarantees stream-ordered execution
- CUDA semantics: operations in same stream execute in order
- Documented safety proofs in $\sim 45,000$ lines of analysis

Q10: What about the vecmem memory alignment bugs?

- Pre-existing bugs in vecmem library, not caused by our changes
- Triggered at specific batch sizes ($N=6, 8, 12$)
- Future work: investigate and fix upstream
- Workaround: avoid those batch sizes in production

Backup: Anticipated Questions (5/6)

Q11: How did you validate at kernel level?

- Used NVIDIA Nsight Compute (NCU) for per-kernel metrics
- Compared occupancy, memory throughput, cache hit rates
- Confirmed baseline fitting kernels at 15% occupancy (85% idle)

Q12: Why do cache hit rates decrease but performance improves?

- Batching increases working set size $\rightarrow$ lower L1 hit rate (48% $\rightarrow$ 43%)
- But better memory coalescing $\rightarrow$ +154% throughput
- Trade-off: bandwidth utilization beats cache efficiency

Q13: What does NCU reveal about future opportunities?

- `fit_forward` at 38%, `fit_backward` at 30% occupancy—headroom remains
- Kernels are memory-bound with scattered access patterns
- Further gains require algorithmic changes to improve locality

Backup: NCU Profiling Details (6/6)

Hardware: NVIDIA GeForce RTX 2080 Ti (CC 7.5, 68 SMs)

Per-Kernel Metrics

Kernel	Occ.	Mem	Grid
<i>Baseline</i>			
propagate	35.0%	224 GB/s	236
find_tracks	21.4%	43.8 GB/s	471
fit_forward	15.4%	61.5 GB/s	112
fit_backward	14.9%	96.8 GB/s	112
<i>Optimized (batch-48)</i>			
propagate	42.8%	264 GB/s	1435
find_tracks	23.7%	57.7 GB/s	2870
fit_forward	38.4%	135.7 GB/s	482
fit_backward	29.9%	245.5 GB/s	482

Cache Performance

Metric	Base	Opt	Δ
L1 Hit Rate	48%	43%	-11%
L2 Hit Rate	78%	77%	-2%

Trade-off Analysis

Cache hit rates decrease due to larger working sets, offset by:

- Better memory coalescing
- Higher bandwidth utilization
- **+154%** memory throughput